

Phase 1 — Modélisation & Générateur

Décomposition des 9 tâches · Notions · Répartition sur 4 personnes

Phase 1 est le socle de tout le projet. Les tâches T1→T6 sont théoriques (contenu du notebook check), T7→T8 sont le code Python, T9 est l'assemblage final. Le générateur produit les instances réutilisées par les phases 2, 3, 4 et 5.

T1

Formaliser $G = (V, A, c)$ — le graphe

modélisation

théorie des graphes

POURQUOI

Tout le projet repose sur une représentation du réseau routier. Sans définition formelle du graphe, aucun algorithme ne peut être spécifié proprement. C'est le langage commun de tout ce qui suit.

COMMENT

Définir formellement chaque composant du triplet $G = (V, A, c)$. Distinguer graphe orienté vs non orienté, complet vs partiel. Introduire la matrice d'adjacence et la matrice des coûts. Justifier pourquoi on travaille sur un graphe complet pour le TSP.

 $V = \{0, 1, \dots, n\}$ (0 = dépôt, 1...n = clients) $A \subseteq V \times V$ (arcs orientés) $c : A \rightarrow \mathbb{R}^+$ (coût euclidien : $c_{ij} = ||\text{coord}_i - \text{coord}_j||_2$)

NOTIONS ABORDÉES

Graphe orienté

Graphe complet

Matrice d'adjacence

Distance euclidienne

Inégalité triangulaire

Clôture métrique

Charge : 1 personne · ~2h**Livrable** : Section 1 du notebook check

T2**Définir les variables de décision x_{ij}^k et t_i^k**

modélisation

PLNE

POURQUOI

Les variables de décision sont ce que le modèle "choisit". Sans elles, impossible d'écrire les contraintes ni la fonction objectif. C'est le pont entre la réalité (tournées) et les mathématiques (programme linéaire en nombres entiers).

COMMENT

Définir x_{ijk} (variable binaire de routage) et t_{ik} (variable continue temporelle). Expliquer pourquoi deux types de variables sont nécessaires : l'une pour le chemin, l'autre pour le planning. Montrer qu'elles sont couplées.

$x_{ijk} \in \{0,1\}$ – 1 si véhicule k emprunte arc $(i \rightarrow j)$
 $t_{ik} \in \mathbb{R}^+$ – heure d'arrivée de k au client i

NOTIONS ABORDÉES

Variable binaire

Variable continue

PLNE

Couplage
routage/ordonnancement

Charge : 1 personne · ~1h30

Livable : Section 2 du notebook check

T3**Écrire la fonction objectif**

modélisation

optimisation

POURQUOI

La fonction objectif définit ce qu'on optimise. Sans elle, le problème n'est pas un problème d'optimisation. Elle quantifie la qualité d'une solution et permet de comparer deux tournées.

COMMENT

Écrire la somme des coûts des arcs utilisés sur tous les véhicules. Expliquer le produit $c_{ij} \cdot x_{ijk}$ (annule les arcs non utilisés). Discuter des variantes : minimiser distance totale vs nombre de véhicules vs retard total.

$\min \sum_k \sum_i \sum_j c_{ij} \cdot x_{ijk}$

NOTIONS ABORDÉES

Fonction objectif

Linéarité

Mono vs multi-critères

Charge : Groupé avec T2 · ~1h

Livable : Section 2 du notebook check

T4

Formaliser les 5 contraintes VRPTW

modélisation

livrable

POURQUOI

Les contraintes distinguent le VRPTW d'un simple TSP. Chaque contrainte traduit une règle métier réelle. C'est la partie la plus dense du livrable check — elle sera évaluée en détail par le tuteur.

COMMENT

Une contrainte par sous-section. Pour chacune : formule, explication en français, exemple numérique concret, et discussion hard vs soft. La contrainte C5 (cohérence temporelle) mérite un développement particulier car elle couple toutes les décisions dans le temps.

C1: $\sum_k \sum_j x_{ijk} = 1$ pour tout $i \neq 0$ (couverture)
C2: $\sum_i x_{ijk} = \sum_i x_{jik}$ pour tout j, k (conservation flux)
C3: $\sum_i q_i (\sum_j x_{ijk}) \leq Q$ pour tout k (capacité)
C4: $a_i \leq t_{ik} \leq b_i$ pour tout i, k (fenêtres temp.)
C5: $t_{ik} + s_i + c_{ij} \leq t_{jk}$ si $x_{ijk}=1$ (cohérence temp.)

NOTIONS ABORDÉES

Contrainte
égalité/inégalité

Conservation de flux

Capacité véhicule

Fenêtres temporelles

Propagation temporelle

Hard vs soft constraints

Charge : 1 personne · ~3h — tâche centrale

Livrable : Section 3 du notebook check

T5

Démontrer NP-difficulté par réduction hamiltonien → TSP

complexité

livrable

POURQUOI

C'est la démonstration théorique qui justifie toute l'architecture du projet. Sans elle, impossible d'expliquer pourquoi on utilise des heuristiques plutôt qu'un algorithme exact. Explicitement demandé dans le livrable check.

COMMENT

Structure en 4 étapes : (1) partir du circuit hamiltonien NP-complet connu, (2) construire en $O(n^2)$ une instance TSP avec poids 0/1, (3) montrer l'équivalence des solutions, (4) conclure par l'absurde. Puis étendre : TSP est un cas particulier de VRPTW → VRPTW est aussi NP-difficile.

$G = (V, E)$ hamiltonien $\rightarrow G' = (V, K_n, w)$
 $w(i, j) = 0$ si (i, j) dans E , $w(i, j) = 1$ sinon
Circuit hamiltonien dans $G \Leftrightarrow$ tournée TSP de coût 0 dans G'
Transformation en $O(n^2)$ $\rightarrow$ TSP est NP-difficile
TSP $\leq_p$ VRPTW $\rightarrow$ VRPTW est NP-difficile

NOTIONS ABORDÉES

NP-complet / NP-difficile

Réduction polynomiale

Circuit hamiltonien

Raisonnement par
l'absurdeCas particulier →
général

Charge : 1 personne · ~2h30

Livrable : Section 4 du notebook check

T6

Calculer la complexité asymptotique de la force brute

complexité

POURQUOI

Demandé explicitement dans le prosit (Agathe veut le temps de calcul théorique). Calculer $O((n-1)!/2)$ et le mettre en perspective avec des valeurs concrètes rend la NP-difficulté convaincante pour la soutenance.

COMMENT

Compter le nombre de permutations de n villes. Expliquer pourquoi on divise par 2 (symétrie) et par n (départ fixé). Calculer pour $n = 10, 15, 20, 50$ et convertir en temps réel à 10^9 op/s. Tracer la courbe $O(n!)$ vs $O(2^n)$ vs $O(n^2)$.

Tournees possibles = $(n-1)! / 2$
 $n=10 \rightarrow 181\,440$ (~0.2 ms)
 $n=15 \rightarrow 43$ milliards (~43 s)
 $n=20 \rightarrow 6 \times 10^{16}$ (~2 000 ans)
 $n=50 \rightarrow 10^{62}$ (age de l'univers $\times 10^{42}$)

NOTIONS ABORDÉES

Notation $O()$
asymptotique

Factorielle

Combinatoire des
permutations

Complexité en temps

Charge : Groupé avec T5 · ~1h**Livable** : Section 4 du notebook check

T7

Coder le générateur d'instances Python

code

Python

POURQUOI

C'est le socle technique de tout le projet. Le générateur produit les instances utilisées par les phases 2, 3, 4 et 5. Une mauvaise conception ici se paie quatre fois. Il doit être propre, documenté, reproductible (seed), et paramétrable.

COMMENT

Générer n coordonnées 2D aléatoires, calculer la matrice de distances euclidiennes, générer des fenêtres temporelles cohérentes ($a_i < b_i$), des demandes q_i et une capacité Q cohérente.

Sortie : dictionnaire Python ou JSON standardisé. Tester sur $n = 5, 10, 20$.

```
instance = {
    'n': n, # nombre de clients
    'coords': ndarray, # (n+1, 2) – depot en [0]
    'dist': ndarray, # (n+1, n+1) matrice distances
    'demands': ndarray, # (n+1,) – demandes clients
    'capacity': float, # Q capacite vehicule
    'time_windows': ndarray, # (n+1, 2) – [a_i, b_i]
    'service_times': ndarray, # (n+1,) – s_i
    'seed': int
}
```

NOTIONS ABORDÉES

numpy random/seed

Distance euclidienne
vectorisée

Sérialisation JSON

Reproductibilité

PEP8

Charge : 1 personne · ~3h — chemin critique**Livrable** : Section 5 du notebook check

T8

Justifier les paramètres du générateur

code

livrable

POURQUOI

L'énoncé demande explicitement de justifier les paramètres des instances. Un générateur sans justification des choix n'est pas scientifique. Il faut argumenter pourquoi ces valeurs de n , Q , bornes des time windows ont été choisies.

COMMENT

Pour chaque paramètre, expliquer la fourchette choisie et pourquoi. Vérifier que les instances sont faisables. S'appuyer sur les benchmarks Solomon (instances de référence VRPTW) pour justifier les ordres de grandeur.

n dans $\{10, 20, 50, 100, 200\}$ – couvre "petit" à "grand"
 $Q = 0.3 \times \text{SUM}(q_i) / m$ – force ~3 véhicules en moyenne
 $[a_i, b_i]$: largeur ~30% horizon – ni trop restrictif ni trop libre
 $s_i \sim U(5, 15)$ minutes – service réaliste livraison

NOTIONS ABORDÉES

Benchmarks Solomon

Faisabilité d'instance

Distribution uniforme

Plan d'expérience
préliminaire**Charge** : Groupé avec T7 · ~1h**Livrable** : Section 5 du notebook check

T9

Rédiger le notebook livrable check

livrable

storytelling

POURQUOI

C'est le livrable évalué par le tuteur. Un travail théoriquement correct mais mal présenté perd des points. L'énoncé insiste sur le storytelling — le notebook doit se lire comme un argument logique du début à la fin.

COMMENT

Assembler T1→T8 dans un ordre narratif cohérent. Chaque cellule code précédée d'une cellule Markdown qui explique le pourquoi. Ajouter une introduction (contexte ADEME), une conclusion (suite du projet), et une bibliographie. Vérifier PEP8 sur tout le code.

Structure du notebook :

1. Contexte et motivation (ADEME, mobilité)
2. Modélisation formelle (T1 → T3)
3. Contraintes VRPTW (T4)
4. Analyse de complexité (T5 → T6)
5. Générateur d'instances (T7 → T8)
6. Conclusion + bibliographie

NOTIONS ABORDÉES

Markdown / LaTeX
Jupyter

Storytelling technique

Bibliographie
scientifique

PEP8

Reproductibilité

Charge : Collectif — 1 coordinateur · ~2h assemblage Livrable : Livrable check final

Répartition recommandée — 4 personnes



Personne 1 — Graphe & Variables

Profil : à l'aise avec les maths formelles

- T1** Formaliser $G = (V, A, c)$ — définition complète, matrice des coûts, distance euclidienne
- T2** Variables de décision x_{ijk} et t_{ik} — justifier chaque variable, expliquer le couplage
- T3** Fonction objectif — écrire et décomposer $\min \sum c_{ij} \cdot x_{ijk}$

~4h · Sections 1 et 2 du notebook



Personne 2 — Contraintes VRPTW

Profil : rigoureux, attentif aux détails

- T4** Formaliser C1 à C5 — une sous-section par contrainte, formule + explication + exemple numérique + hard/soft

~3h · Section 3 du notebook — tâche la plus dense théoriquement



Personne 3 — Complexité & NP

Profil : logique, aime la démonstration

- T5** Réduction hamiltonien $\rightarrow$ TSP $\rightarrow$ VRPTW — démonstration complète en 4 étapes
- T6** Complexité $O((n-1)!/2)$ — table de valeurs, courbe, mise en perspective

~3h30 · Section 4 du notebook



Personne 4 — Code & Assemblage

Profil : Python, organisé

- T7** Générateur Python — coords, distances, time windows, capacités, JSON
- T8** Justification des paramètres — benchmarks Solomon, faisabilité
- T9** Assemblage final du notebook — intégrer P1+P2+P3, storytelling, biblio, PEP8

~5h · Sections 5 et 6 + coordination finale

Point de synchronisation recommandé

Avant T9, les 4 personnes se réunissent 30 min pour harmoniser les notations. Si P1 écrit $V = \{0, \dots, n\}$ et P2 écrit $V = \{1, \dots, n+1\}$, le notebook sera incohérent. Décider ensemble : notation des sommets, nom des variables, symboles utilisés. P4 intègre ensuite en version unifiée.